ETX Signal-X

Daily Intelligence Digest

Thursday, March 19, 2026

10	11
ARTICLES	TECHNIQUES

Article 1

How I Chained a Simple Text Injection with OAuth Misconfiguration on NASA

Source: securitycipher

Date: 28-Jun-2025

URL: https://insomnia-x.medium.com/how-i-chained-a-simple-text-injection-with-oauth-misconfiguration-on-nasa-497223652bde

T1 Text Injection in Username Reflected Across OAuth-Connected Subdomains

💉 Payload

```
Login Failed. Please login at malicious-research.nasa.gov/login.
```

⚔️ Attack Chain

1. Register a new account on `redacted.nasa.gov` with the username set to:
2. Complete the registration and login process.
3. Initiate an OAuth login flow on `redacted-research.nasa.gov` using the account created in step 1.
4. The username is reflected in the welcome message on `redacted-research.nasa.gov`, e.g.,
5. Victim sees the message and is tricked into visiting the attacker's phishing site at `malicious-research.nasa.gov/login`.

🔍 Discovery

Manual exploration of the registration and login flows on `redacted.nasa.gov` revealed lack of input validation on the username field. The reflection was confirmed by observing the welcome message on the OAuth-connected subdomain after login.

🔓 Bypass

N/A (No input validation or sanitization on username field; no WAF or filter bypass required.)

🔗 Chain With

Can be chained with OAuth misconfiguration (see Technique 2) to force login victims into attacker-controlled accounts and deliver targeted phishing payloads.

T2 OAuth Forced Login via Missing State Parameter (CSRF on OAuth Flow)

💉 Payload

```
https://redacted-research.nasa.gov/login?code=xyz
```

⚔️ Attack Chain

1. Attacker creates an account on `redacted.nasa.gov` with a chosen username (see Technique 1 for phishing payload).
2. Attacker initiates an OAuth login flow on `redacted-research.nasa.gov` and captures the resulting `code` parameter (e.g., `code=xyz`).
3. Attacker crafts a link to `https://redacted-research.nasa.gov/login?code=xyz` and sends it to the victim.
4. Victim clicks the link, and due to the absence of the `state` parameter in the OAuth flow, the victim is logged into the attacker's account on `redacted-research.nasa.gov` (forced login/CSRF).
5. The victim sees the attacker's username reflected (e.g., the phishing message from Technique 1).

🔍 Discovery

Review of OAuth login requests in Burp Suite revealed the absence of the `state` parameter, indicating a missing CSRF protection in the OAuth flow.

🔓 Bypass

OAuth implementation did not require or validate the `state` parameter, allowing CSRF and forced login attacks without any additional bypass.

🔗 Chain With

Can be chained with text injection (Technique 1) to deliver highly convincing phishing payloads to victims via forced login, increasing the impact from low to high severity.

Article 2

200$ by Tricking a Global Music App with One Line of Code

Source: securitycipher

Date: 17-Apr-2025

URL: https://myselfakash20.medium.com/200-by-tricking-a-global-music-app-with-one-line-of-code-de2f4ab3cd4a

⚠️ No actionable techniques found

Article 3

How I Found a Critical Biometric 2FA Bypass... and Lost the Bounty

Source: securitycipher

Date: 13-Jan-2026

URL: https://letchupkt.medium.com/how-i-found-a-critical-biometric-2fa-bypass-and-lost-the-bounty-9c38441640c4

T1 Biometric 2FA Bypass via Hardcoded Superadmin Key in Public JavaScript

💉 Payload

```
https://third-party-provider.com/biometric_2fa/service/user/matchFingerPrint?key=superadmin
```

⚔️ Attack Chain

1. Identify the ARCON PAM login portal (e.g., pam.target-x.com) and open the login page.
2. Use browser DevTools to inspect loaded JavaScript files, focusing on custom or non-standard scripts (e.g., /resources/js/custom/LegendScript.js).
3. Locate the hardcoded API key (e.g., key=superadmin) for the third-party biometric authentication provider embedded in the JS file.
4. Craft direct API requests to the third-party biometric endpoint using the exposed superadmin key (e.g., matchFingerPrint?key=superadmin).
5. Observe that the endpoint responds (200 OK or 400 Bad Request), confirming the key is live and accepted.
6. Leverage the superadmin key to forge or manipulate biometric verification responses, effectively bypassing biometric 2FA for any admin-level account on the PAM portal.

🔍 Discovery

Manual static analysis of client-side JavaScript files loaded by the ARCON PAM login page, specifically targeting custom-named scripts likely to contain business logic or secrets (e.g., LegendScript.js). The suspicious variable name and unusual script location triggered deeper inspection.

🔓 Bypass

By using the hardcoded superadmin key, an attacker can directly interact with the biometric verification API, forging positive responses to biometric checks and bypassing the intended 2FA mechanism without needing valid biometric data.

🔗 Chain With

Use the bypassed PAM access to pivot into internal infrastructure, escalate privileges, or access further sensitive systems. Combine with session fixation, IDOR, or other authentication flaws for lateral movement or persistence. Potential for supply chain compromise if the third-party biometric provider is used by other clients.

T2 Targeted JavaScript Recon for Secret Discovery in Enterprise Portals

💉 Payload

```
/resources/js/custom/LegendScript.js
```

⚔️ Attack Chain

1. Enumerate subdomains and identify privileged access portals (e.g., PAM, admin, sso).
2. Open the portal and use browser DevTools to enumerate all loaded JavaScript assets.
3. Prioritize manual review of custom-named or non-minified scripts (e.g., LegendScript.js, custom.js, test.js) over standard libraries.
4. Search for hardcoded credentials, API keys, or sensitive endpoints within these scripts.
5. Extract any secrets or endpoints for further exploitation (e.g., direct API calls, authentication bypass, privilege escalation).

🔍 Discovery

Manual inspection of loaded JavaScript files, focusing on those with non-standard names or located in custom directories. The presence of a script named LegendScript.js in a /custom/ directory was a red flag, leading to the discovery of embedded secrets.

🔓 Bypass

By skipping generic libraries and focusing on custom scripts, attackers can find secrets and logic errors missed by automated scanners and static analysis tools. This approach bypasses defenses that rely on obfuscation or assume secrets are only exposed via API responses.

🔗 Chain With

Use discovered secrets to attack third-party services, internal APIs, or authentication flows. Combine with SSRF, IDOR, or business logic flaws for deeper compromise. Automate JavaScript recon across all privileged portals for organization-wide coverage.

Article 4

CVE-2025–29927: Explotando un middleware vulnerable paso a paso

Source: securitycipher

Date: 20-Sep-2025

URL: https://gorkaaa.medium.com/cve-2025-29927-explotando-un-middleware-vulnerable-paso-a-paso-e65a2a58f735

⚠️ No actionable techniques found

Article 5

$256 Bounty : XSS via Web Cache Poisoning in Discourse

Source: securitycipher

Date: 13-May-2025

URL: https://infosecwriteups.com/256-bounty-xss-via-web-cache-poisoning-in-discourse-594d5961555e

T1 XSS via X-Forwarded-Host Header Injection in Discourse Font Preload URLs

💉 Payload

```
GET /?xx HTTP/1.1
Host: meta.discourse.org
X-Forwarded-Host: cacheattack'"><script>alert(document.domain)</script><script> scriptalert
(document.domain)</script> script
```

⚔️ Attack Chain

1. Identify a Discourse-based target (e.g., meta.discourse.org, discourse.mozilla.org, help.nextcloud.com).
2. Analyze the template rendering (specifically `_special_font_face.html.erb`) to confirm user input from the Host header is reflected unsanitized in HTML (e.g., in font preload URLs).
3. Send a request with a malicious `X-Forwarded-Host` header containing a JavaScript payload as shown above.
4. Observe the rendered HTML: the payload is injected into the `href` attribute of a `<link rel="preload">`, causing the browser to parse and execute the script.
5. The response is cached for 60 seconds, so subsequent users receive the poisoned page, resulting in a temporary stored XSS.

🔍 Discovery

Manual source review and template analysis of Discourse instances, focusing on the use of `.html_safe` with user-controlled headers. Payload testing in font preloading URLs exposed the injection point.

🔓 Bypass

Exploits unsafe use of `.html_safe` and lack of sanitization on `X-Forwarded-Host`. Leverages web cache behavior to persist the payload for other users (Web Cache Deception). Works even if Host header is validated, as X-Forwarded-Host is often trusted by backend frameworks.

🔗 Chain With

Can be chained with session hijacking, cookie theft, or further privilege escalation if the XSS lands on authenticated pages. Potential to combine with CDN cache poisoning for longer-lived or wider impact if CDN caches based on header combinations.

T2 Web Cache Deception Amplifying Header-Based XSS to Stored-Like XSS

💉 Payload

```
curl -H "X-Forwarded-Host: evil.com'><script>alert(1)</script>" https://target.site/<script
> scriptalert(1)</script> script
```

⚔️ Attack Chain

1. Identify a target using a CDN or reverse proxy that caches HTML responses (e.g., CloudFront, Fastly).
2. Inject a malicious `X-Forwarded-Host` header with a JavaScript payload in a request to a cacheable endpoint.
3. Confirm that the server reflects the header into a dynamic HTML context (e.g., font preload link, meta tag).
4. Observe that the response is cached for a period (e.g., 60 seconds) and served to subsequent users without their involvement.
5. Other users receive and execute the XSS payload, resulting in a stored-like XSS attack.

🔍 Discovery

Testing cache behavior by varying headers (Accept, Accept-Encoding, X-Forwarded-Host) and monitoring if poisoned responses persist for other users. Focused on endpoints where header-based reflection occurs in HTML.

🔓 Bypass

Exploits the fact that cache keys may include or ignore certain headers, allowing an attacker to poison the cache for a wider audience. Works even if the application is not vulnerable to classic stored XSS, as the cache stores the poisoned response.

🔗 Chain With

Can be used to deliver XSS to privileged users (admins, moderators) who load the poisoned page within the cache window. May be chained with CSRF or session fixation if the cache persists authentication state.

Article 6

Crypto bounty program got me $500 — Rate Limit Bypass

Source: securitycipher

Date: 06-May-2024

URL: https://mo9khu93r.medium.com/crypto-bounty-program-got-me-500-rate-limit-bypass-d573f7b7d390

T1 Rate Limit Bypass on Password Reset via Trailing Space in Email

💉 Payload

```
POST /forgot-password HTTP/1.1
Host: [target]
Content-Type: application/x-www-form-urlencoded
Content-Length: [length]

email=email@gmail.com
```

⚔️ Attack Chain

1. Intercept a valid password reset request (e.g., to `/forgot-password`) with the target user's email (e.g., `email@gmail.com`).
2. Send the request 5 times with the exact email; after 5 attempts, the account is locked for 3 minutes (rate limit enforced).
3. Modify the `email` parameter by appending a single space after the email address (e.g., `email@gmail.com `).
4. Send the modified request 5 more times; each request triggers a password reset email, and after 5 attempts, the rate limit is again enforced for this variant.
5. Repeat step 3, incrementing the number of trailing spaces after every 5 attempts (e.g., `email@gmail.com `, `email@gmail.com `, etc.), to continuously bypass the rate limit and trigger unlimited password reset emails.

🔍 Discovery

Initial attempts to bypass the rate limit using null bytes (`\0`, `\x00`), newline (`\n`), and URL encoding (`%0`) failed. The researcher then hypothesized that the backend comparison might not trim whitespace, so tried appending a space, which successfully bypassed the rate limit logic.

🔓 Bypass

The rate limit mechanism keyed off the raw email string without normalization (no trimming of whitespace). Each unique variant with a different number of trailing spaces was treated as a separate identity, allowing the attacker to reset the rate limit counter per variant.

🔗 Chain With

Can be chained with account lockout/DoS attacks by flooding a target user with password reset emails, potentially leading to account lockout or email spam. May be combined with other logic flaws where email normalization is not enforced (e.g., registration, login, or email change endpoints) to create further inconsistencies or bypasses.

Article 7

The OTP That Told on Itself—How I Bypassed Email Verification with One Wrong Code

Source: securitycipher

Date: 05-Oct-2025

URL: https://msnrasel1.medium.com/the-otp-that-told-on-itself-how-i-bypassed-email-verification-with-one-wrong-code-67236eb803a1

T1 OTP Leakage via Duplicate Parameter in Email Verification

💉 Payload

```
POST /Account/VerifyEmail HTTP/1.1
Host: 3eyedraven.com
Content-Type: application/x-www-form-urlencoded

Email=testuser@example.com&ConfirmCode=395821&ConfirmCode=123456& hundreads of more paramet
ers.
```

⚔️ Attack Chain

1. Register a new account at https://3eyedraven.com/Account/Register with any email address.
2. Receive a 6-digit OTP via email (e.g., 395821).
3. Enter any random, incorrect OTP (e.g., 123456) in the verification form.
4. Intercept the verification request using Burp Suite or similar proxy.
5. Observe that the request contains two `ConfirmCode` parameters: the first is the real OTP (395821), the second is the user-supplied value (123456).
6. Copy the real OTP value from the intercepted request.
7. Resend the request with the correct OTP to bypass email verification without accessing the inbox.

🔍 Discovery

Testing the email verification flow with an intentionally wrong OTP and intercepting the request revealed the presence of two `ConfirmCode` parameters, with the first containing the actual OTP sent by the server.

🔓 Bypass

The server leaks the real OTP in a duplicate parameter, allowing an attacker to extract the OTP from the request itself and bypass the intended verification process.

🔗 Chain With

Can be chained with automated account creation for mass registration attacks, as email verification is rendered ineffective. May facilitate further account takeover or privilege escalation if similar parameter leakage exists in other authentication flows.

Article 8

Account Deletion Security Pitfalls — A Bug Bounty Case Study

Source: securitycipher

Date: 14-Aug-2025

URL: https://sohanxp56.medium.com/account-deletion-security-pitfalls-a-bug-bounty-case-study-8ab8fe7ac8f5

T1 Account Deletion via Bearer Token Only (No Secondary Verification)

💉 Payload

```
DELETE /user HTTP/1.1
Host: target.tld
Authorization: Bearer <victim_token>
Content-Type: application/json
```

⚔️ Attack Chain

1. Attacker obtains a valid bearer token for the victim (e.g., via XSS, phishing, or token leak).
2. Attacker sends a DELETE request to the /user endpoint with the victim's bearer token in the Authorization header.
3. The endpoint deletes the account associated with the provided token, without requiring any user ID, password re-entry, or multi-factor authentication.

🔍 Discovery

Manual review of the account deletion API in a sandbox environment. The researcher noticed that the endpoint did not require a user ID in the URL or body and succeeded with only the bearer token. Swapping the token to another valid user's token deleted that account instantly.

🔓 Bypass

No secondary verification (e.g., password, MFA, or CSRF token) is required. The only check is possession of a valid bearer token. No confirmation or deletion delay is implemented, and no notification is sent to the user.

🔗 Chain With

Combine with any bearer token theft method (XSS, IDOR, session fixation, etc.) for full account destruction. Can be used for targeted DoS by mass-deleting user accounts if tokens are harvested at scale. Can disrupt business workflows relying on bearer tokens for automation.

Article 9

How I Earned $8947 bounty for Remote Code Execution via a Hijacked GitHub Module

Source: securitycipher

Date: 28-Apr-2025

URL: https://nvk0x.medium.com/how-i-earned-8947-bounty-for-remote-code-execution-via-a-hijacked-github-module-91c4a4b63255

T1 Go Module Namespace Hijacking for Remote Code Execution

💉 Payload

```
module github.com/Redacted-examples/somepath/server/go
```

⚔️ Attack Chain

1. Identify Go module import paths in public repositories belonging to the target (e.g., `go.mod` files referencing `github.com/Redacted-examples/...`).
2. Verify if the referenced GitHub namespace (e.g., `Redacted-examples`) is unclaimed or available for registration.
3. Register the unclaimed GitHub account (`github.com/Redacted-examples`).
4. Create a repository with the exact path expected by the Go module (e.g., `somepath/server/go`).
5. Upload a malicious Go source file as a proof-of-concept that executes arbitrary code when imported/built.
6. Wait for the target's CI/CD or developer environment to resolve the dependency, resulting in the attacker's code being fetched and executed, leading to RCE.

🔍 Discovery

Manual code review of public GitHub repositories for third-party module references. The researcher noticed a Go module import path referencing an unclaimed GitHub account, indicating a potential namespace hijack vector.

🔓 Bypass

No authentication or ownership validation is performed by Go modules when resolving dependencies from GitHub. The module system trusts the namespace and path, so registering the unclaimed account is sufficient to inject code.

🔗 Chain With

Pivot to internal network or production servers if the compromised code is executed in privileged CI/CD environments. Use as a supply chain attack vector to implant persistent backdoors or exfiltrate secrets during build/deploy phases. Combine with dependency confusion or typosquatting for broader impact across organizations using similar import patterns.

Article 10

How I Found a Way to Prolong Password Reset Code Expiry

Source: securitycipher

Date: 13-May-2025

URL: https://infosecwriteups.com/how-i-found-a-way-to-prolong-password-reset-code-expiry-6214391023de

T1 Prolonging Password Reset Code Expiry via Repeated Requests

💉 Payload

```
POST /password-reset/request-code HTTP/1.1
Host: target.com
Content-Type: application/json

{
  "email": "victim@target.com"
}
```

⚔️ Attack Chain

1. Initiate a password reset for a target account (e.g., victim@target.com) to receive a 6-digit verification code via email.
2. Before the code expires (e.g., within the 1-hour window), send another password reset request for the same email address.
3. Observe that the same verification code is re-issued and its expiry timer is reset (e.g., back to 1 hour from the latest request).
4. Repeat step 2 as many times as desired, each time before expiry, to keep the same code alive indefinitely.

🔍 Discovery

Manual testing of the password reset flow revealed that the code did not change on repeated requests and the expiry timer was reset rather than issuing a new code or invalidating the old one. The researcher noticed the code's lifetime could be artificially extended by repeatedly requesting it before expiry.

🔓 Bypass

The intended security control (short-lived code) is bypassed by exploiting the logic that resets the expiry timer on every request for the same email, instead of generating a new code or invalidating the old one. No rate-limiting or code invalidation logic is enforced per user/email.

🔗 Chain With

Greatly increases the window for brute-forcing the 6-digit code, as the attacker can keep the code alive for hours or days. Can be combined with credential stuffing or enumeration attacks, as the reset code remains valid for extended periods. May enable persistent account takeover if the code is intercepted or guessed.